GNG5140

AI PROJECT ORGANIZATION

Submitted by

Group - AI PROJECT ORGANIZATION

Aoqi Liu, 300507810

Lizzy-Preiz, 300457263

Zhe Wang, 300497541

Zhenyan Xing, 300321940

Feb 3rd, 2026

University of Ottawa

Abstract

Shared Services Canada (SSC) provides critical digital infrastructure and IT services to Canadian federal government organizations, supporting thousands of project managers who coordinate complex initiatives.
These project managers face significant challenges in managing high-volume, fragmented information streams, including emails, meetings, documents and urgent escalations while maintaining compliance with strict government data governance requirements. This design research report presents a systematic analysis of the operational challenges faced by SSC project managers and establishes the foundation for developing an AI-based project management intelligence solution. Through design status review, technical and user benchmarking, and prototype test planning, this study identifies key user needs centered on daily work orchestration, information staging from unstructured artifacts, decision support for risk management and technical diagram interpretation. The project combines existing project management and AI-assisted workflow tools, establishes measurable performance metrics derived from user needs, and develops target specifications for a prototype. The findings indicate that current solutions inadequately address the cognitive overload experienced by federal project managers, presenting an opportunity for an intelligent triage and decision-support layer that can transform fragmented inputs into actionable insights while respecting government data constraints.

Table of Contents

Table of Contents 3

List of Figures 6

List of Tables 7

List of Acronyms ..... **Error! Bookmark not defined.**

2 Introduction 8

2.1 Overall Structure 9

2.2 Subfunction Descriptions and linkage 10

2.3 User Interface Flows 12

2.3.1 Chatbot flow 12

2.3.2 Tasks flow 13

2.4 Testing Interface 13

3 Prototype and Test 16

3.1 Test data and set up 16

3.1.1 Testdata 16

3.1.2 Hardware and Software Environment 17

3.2 M1 Ingestion Pipeline Prototype 18

3.2.1 Purpose 18

3.2.2 Design & Implementation 18

3.2.3 Test Results 19

3.2.4 Analysis 21

3.3 Risk Identifier Prototype 21

3.3.1 Purpose 21
3.3.2 Design and Implementation 22
3.3.3 Test Results 22
3.3.4 Analysis 22
3.4 Priority Ranker Prototype 23
3.4.1 Purpose 23
3.4.2 Design and Implementation 23
3.4.3 Results 24
3.4.4 Analysis 24
3.5 Drift Detection Prototypes 25
3.5.1 Purpose 25
3.5.2 Design and Implementation 26
3.5.3 Results 27
3.5.4 Analysis 27
3.6 Cross-Module Summary and Integration Testing 28
3.6.1 Overall Score Summary 28
3.6.2 Known Issues Summary 28
4 Updated Prototype Test Plan 29
4.1 Current Status 29
4.2 Project Plan Updates 30
4.3 Task Assignments 31
Time Schedule ..... **Error! Bookmark not defined.**
4.4 Contingency Plan Updates 32

4.4.1 API or model instability 32
4.4.2 Transition risk from test UI to new UI 32
4.4.3 Security requirement escalation from clients 33
4.4.4 Drift module confidence ceiling risk 33
4.4.5 Tencent local anonymization model substitution risk/opportunity 33
5 Bill of Materials 33
5.1.1 API 33
5.1.2 Server renting 34
5.1.3 Electricity 34
6 Conclusions and Recommendations for Future Work 35
7 Bibliography 37

List of Figures

Figure 1 Problem-to-Solution Mapping 9

Figure 2 System Architecture 10

Figure 3 Subfunction Interaction Diagram 11

Figure 4 chat flow 12

Figure 5 taska flow 13

Figure 6 demo 1 14

Figure 7 demo 2 14

Figure 8 demo 3 15

Figure 9 demo 4 15

Figure 10 demo 5 16

Figure 11 Current api cost 34

List of Tables

Table 1 Module Responsibilities and Interfaces 10

Table 2 Test Hardware Specifications 17

Table 3 Model Tiers Used in Testing 17

Table 4 Ingestion Prototype Configurations 19

Table 5 IWngestion Module: Full Results by Configuration 19

Table 6 Ingestion Module: Timing Comparison 20

Table 7 Risk Identifier Module: Results by Configuration 22

Table 8 Priority Ranker Module: Results by Configuration 24

Table 9 Drift Detection Prototype Configurations 26

Table 10 Drift Detection Scenario Types 26

Table 11 Drift Detection Module: Results by Configuration 27

Table 12 Overall Cross-Module Test Results 28

1 Introduction

Project-based teams in SSC-like environments manage critical information across fragmented channels, including emails, meeting transcripts, local documents, and planning tools. This fragmentation creates high coordination effort, inconsistent situational awareness, and delayed identification of delivery and compliance risks. In our earlier analysis, we translated these pain points into four user stories: Daily Project Manager support (US1), Project Risk Checker (US2), Scope Creep Prevention (US3), and Legacy Data Management (US4). Existing tools such as OpenClaw demonstrate promising language understanding and extraction capabilities, but still lack key project management features required for end-to-end operational support, including robust evidence-based prioritization, governance-friendly controls, and integrated decision workflows.

To address this gap, our team designed an AI-powered project management workflow that unifies multi-source project data through a modular architecture. The solution includes an ingestion gateway and five core intelligence services: ingestion pipeline, priority ranker, risk engine, daily planner, and drift detector. This deliverable reports the initial prototype implementation and testing progress, with emphasis on validating assumptions against Deliverable B target requirements using structured expected-versus-actual results. It also documents design adjustments made during execution, including contingency responses to technical constraints and strategic scope decisions. Finally, we present an updated prototype test plan, task ownership model, and procurement-oriented BOM to support the next development cycle toward a robust, evidence-driven project assistant.

2 Global Solution Concept

In this project, the team proposes an AI-powered project management workflow platform that consolidates fragmented information from emails, meeting notes, and local project files. This concept is directly aligned with the four user stories defined in the presentation: US1 (Daily Project Manager), US2 (Project Risk Checker), US3 (Scope Creep Prevention), and US4 (Legacy Data Management). Compared with existing tools, our approach emphasizes converting multi-source project inputs into actionable outputs through one integrated pipeline, reducing manual consolidation effort while improving risk and drift visibility. We changed the architecture a little with one more module of daily planner after got inspired by the lecture.

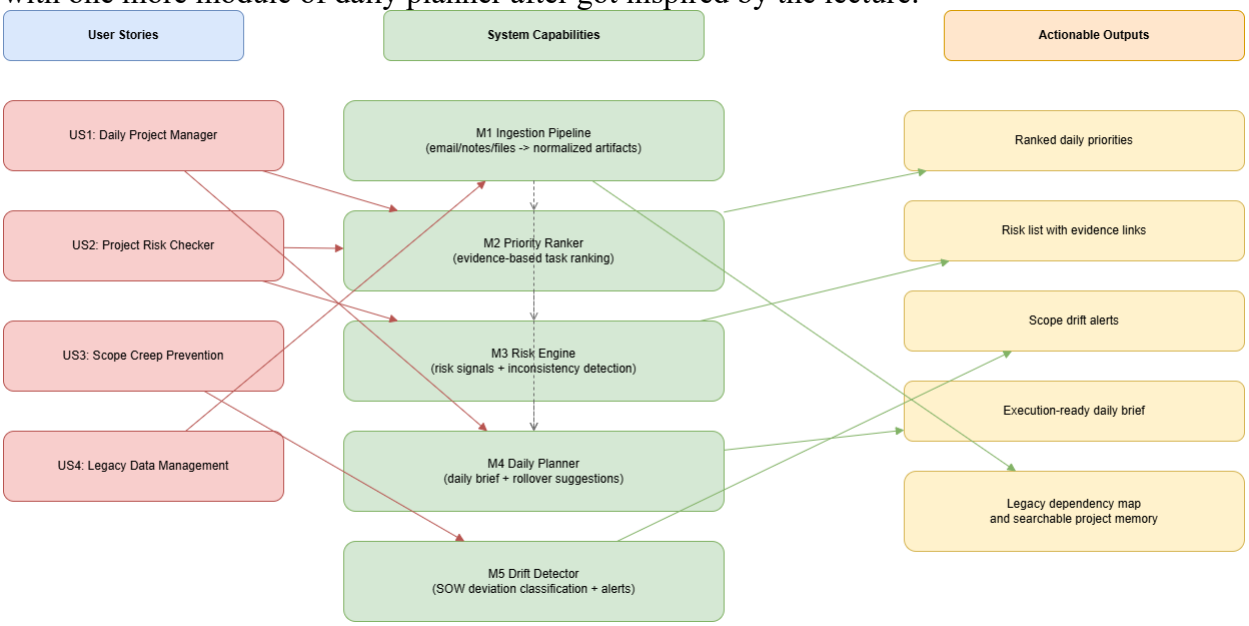


Figure 1 Problem-to-Solution Mapping

2.1 Overall Structure

The architecture follows a layered design. The input layer ingests data from Bot Mailbox (CC/BCC emails), local files (PDF/DOCX/XLSX) and pasted text (for example, meeting minutes). Data is processed by the Ingestion Gateway through format detection, content extraction, normalization and storage. Normalized artifacts are then processed by five core modules in the AI Processing Layer: M1 Ingestion Pipeline, M2 Priority Ranker, M3 Risk Engine, M4 Daily Planner, and M5 Drift Detector. The data and support layer is backed by PostgreSQL, Redis, audit logging, and bilingual (EN/FR) support.

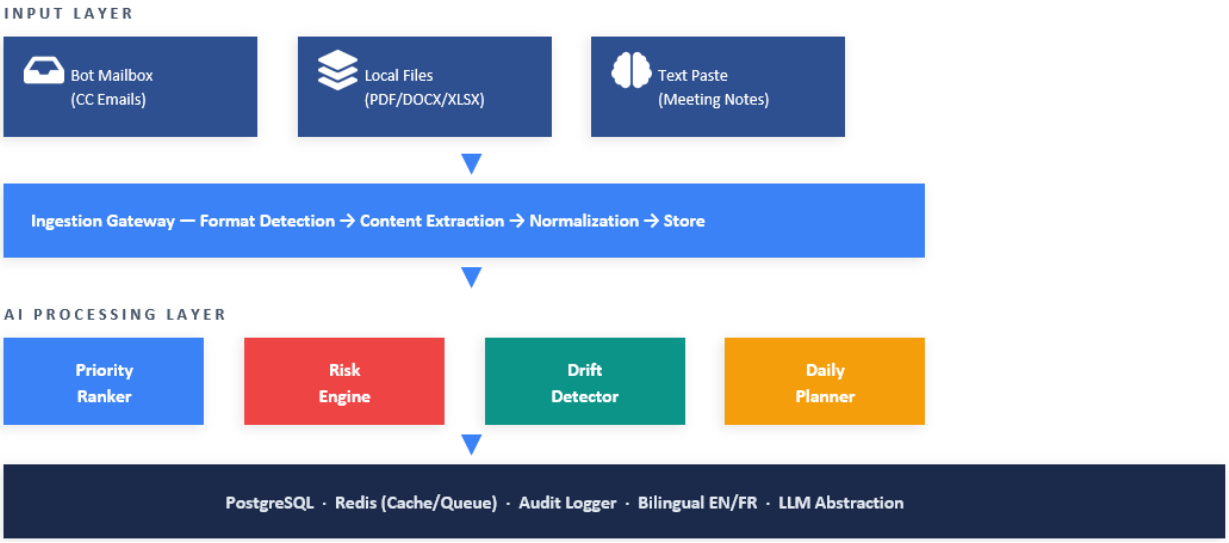


Figure 2 System Architecture

2.2 Subfunction Descriptions and linkage

In terms of module linkage, the Ingestion Pipeline is the main entry point that transforms unstructured content into standardized artifacts; the Risk Engine identifies hidden risks and cross-document inconsistencies; the Drift Detector compares new requests against the SOW baseline; the Priority Ranker produces evidence-based ranking; and the Daily Planner assembles ranked tasks and risk signals into execution-ready daily planning output. This modular structure supports independent module iteration and cross-module integration testing, which matches the team strategy shown in the presentation: deliver a functional prototype first, then expand governance and advanced controls incrementally.

Table 1 Module Responsibilities and Interfaces

Module	Primary Inputs	Primary utputs	Mapped User Stories
M1 Ingestion Pipeline	Emails/Documents/Text	Structured artifacts and extracted tasks	US1, US2, US3, US4
M2 Priority Ranker	Task pool, deadlines, risk signals	Ranked task list	US1, US2

Module	Primary Inputs	Primary utputs	Mapped User Stories
M3 Risk Engine	Artifact content and context	Risks, evidence links, risk scores	US2
M4 Daily Planner	Ranked tasks and status history	Daily brief and rollover suggestions	US1
M5 Drift Detector	SOW baseline and new requests	Drift alerts and drift type classification	US3

From a technology perspective, the implementation uses FastAPI for backend services, PostgreSQL and Redis for data and performance support, and an LLM abstraction layer to switch across model tiers based on quality, latency, and cost constraints. The frontend dashboard provides a unified interaction surface for project stakeholders. This stack balances delivery speed, modularity, and scalability, making it suitable for prototype validation within the course timeline.

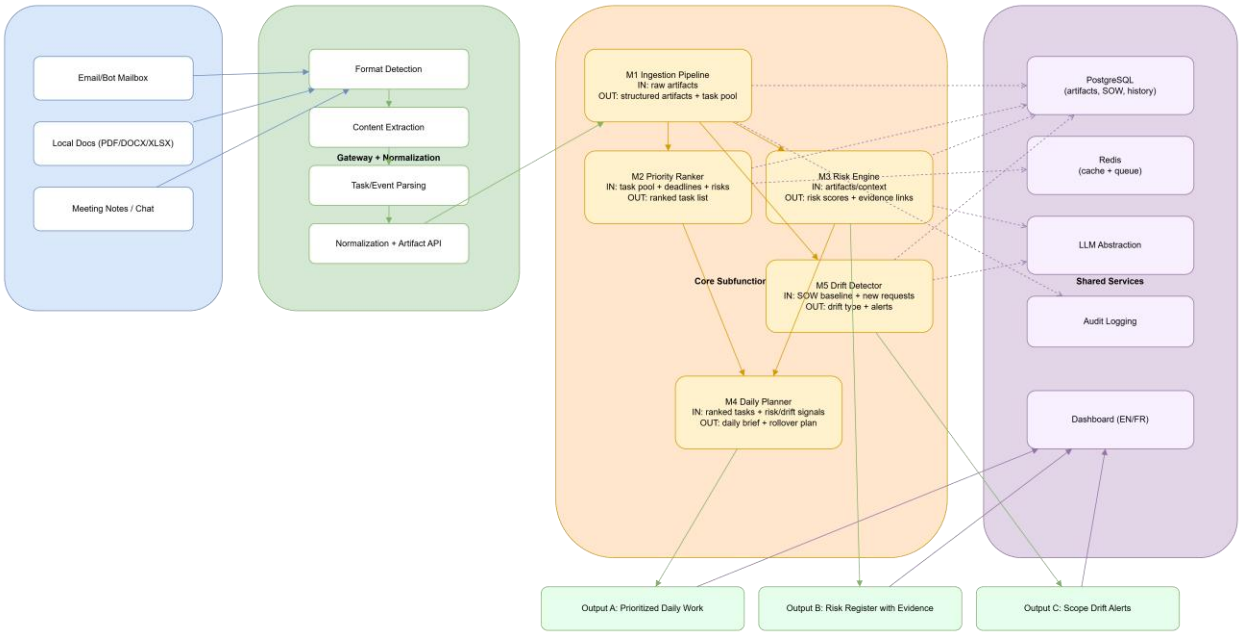


Figure 3 Subfunction Interaction Diagram

2.3 User Interface Flows

For this stage, we are justifying the function of the modules, so the ui flows are not completed and currently we are providing the function flows of the program.

2.3.1 Chatbot flow

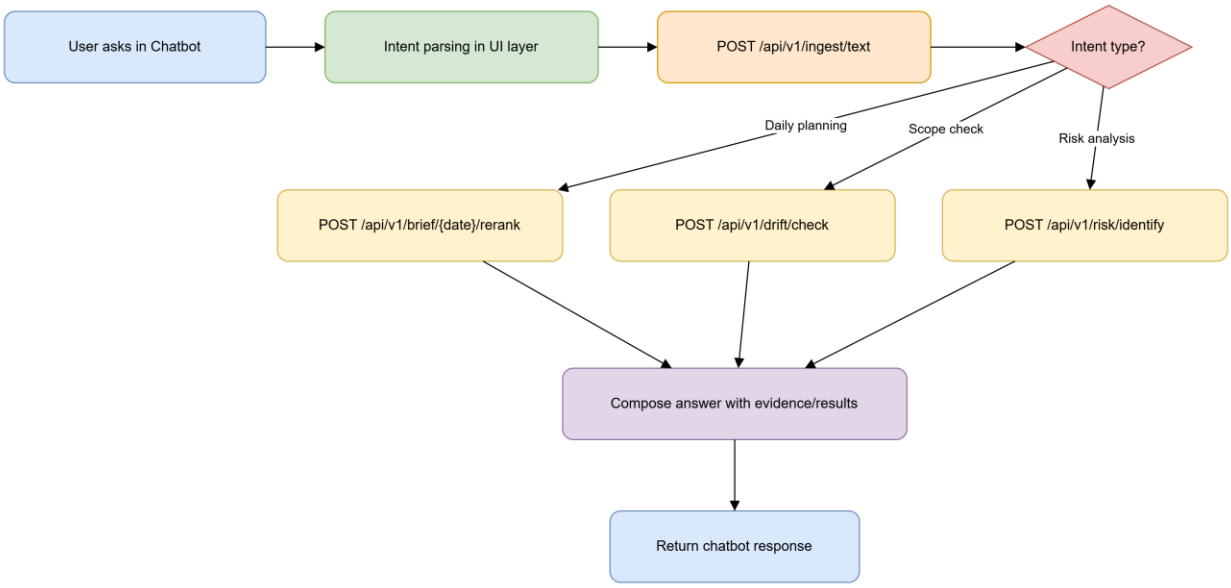


Figure 4 chat flow

This figure explains how the chatbot acts as an orchestration layer over backend APIs. After user input, the UI performs intent parsing and sends normalized text to POST /api/v1/ingest/text. The request then branches by intent: risk analysis (POST /api/v1/risk/identify), scope drift checking (POST /api/v1/drift/check), or daily reprioritization (POST /api/v1/brief/{date}/rerank). The final response is composed from module outputs and supporting evidence references, so answers remain traceable to system results.

2.3.2 Tasks flow

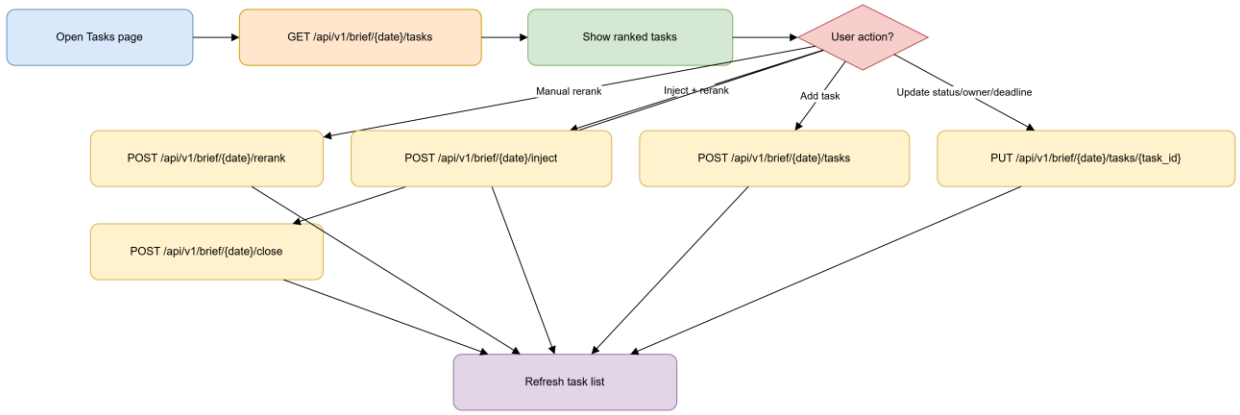


Figure 5 taska flow

This flow describes how users interact with the task management interface during daily execution. The page loads ranked tasks through GET /api/v1/brief/{date}/tasks, then supports update, add, inject-and-rerank, manual rerank, and close-day actions via dedicated endpoints. All action branches converge to a refresh step to keep UI state synchronized with backend ranking results. This interaction loop directly supports US1 by enabling continuous task control and mid-day prioritization updates.

2.4 Testing Interface

Currently since we are dealing with functional module, we implemented an testing interface instead of building a proper web front. Otherwise, it will be very hard to verify the errors. And here are the figures of current testing interface.

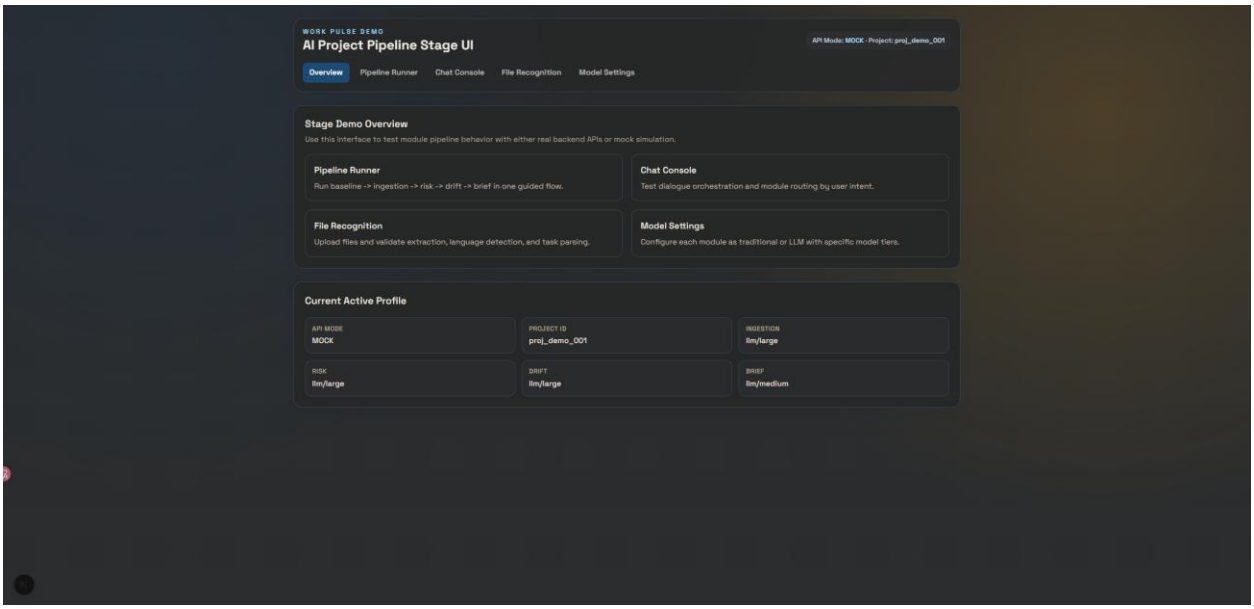


Figure 6 demo 1

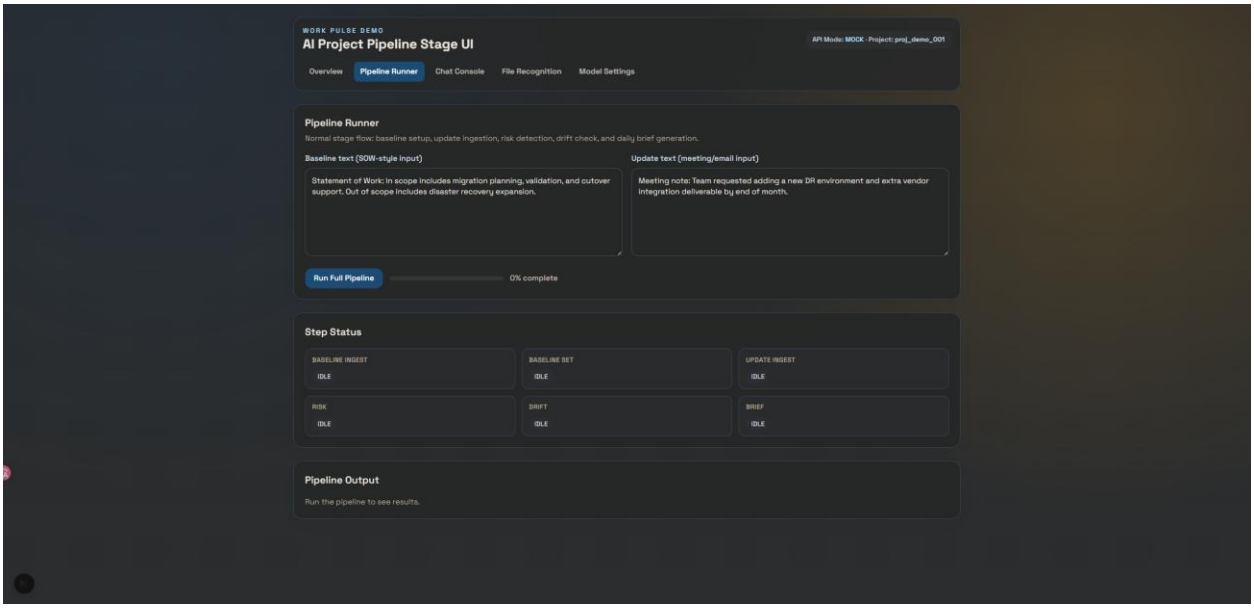


Figure 7 demo 2

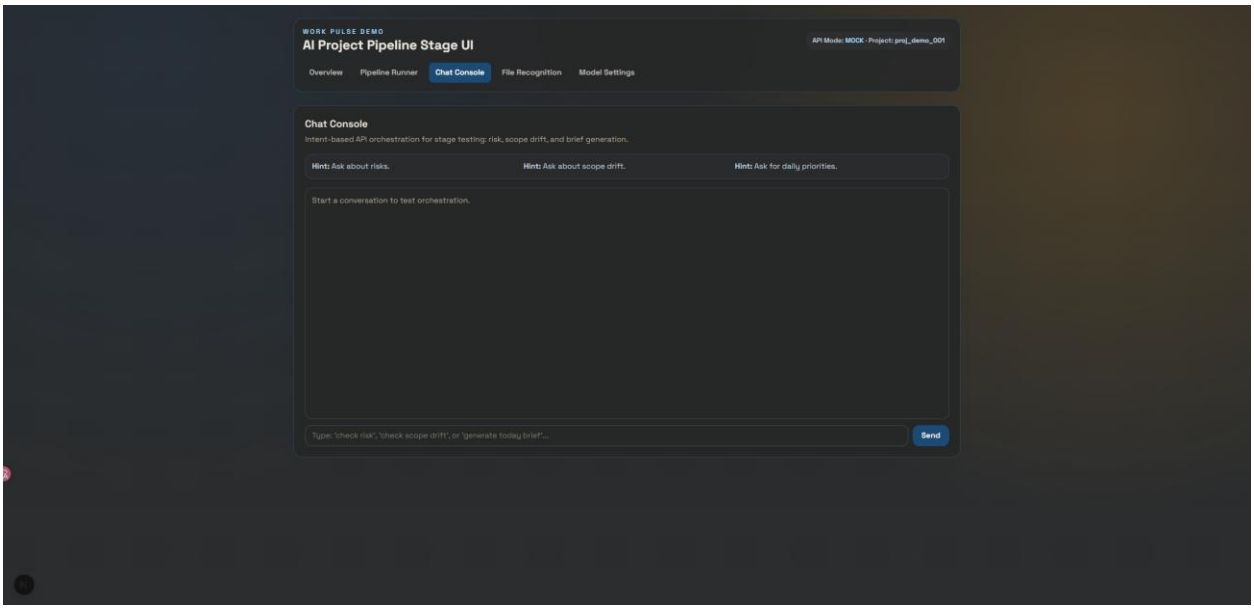


Figure 8 demo 3

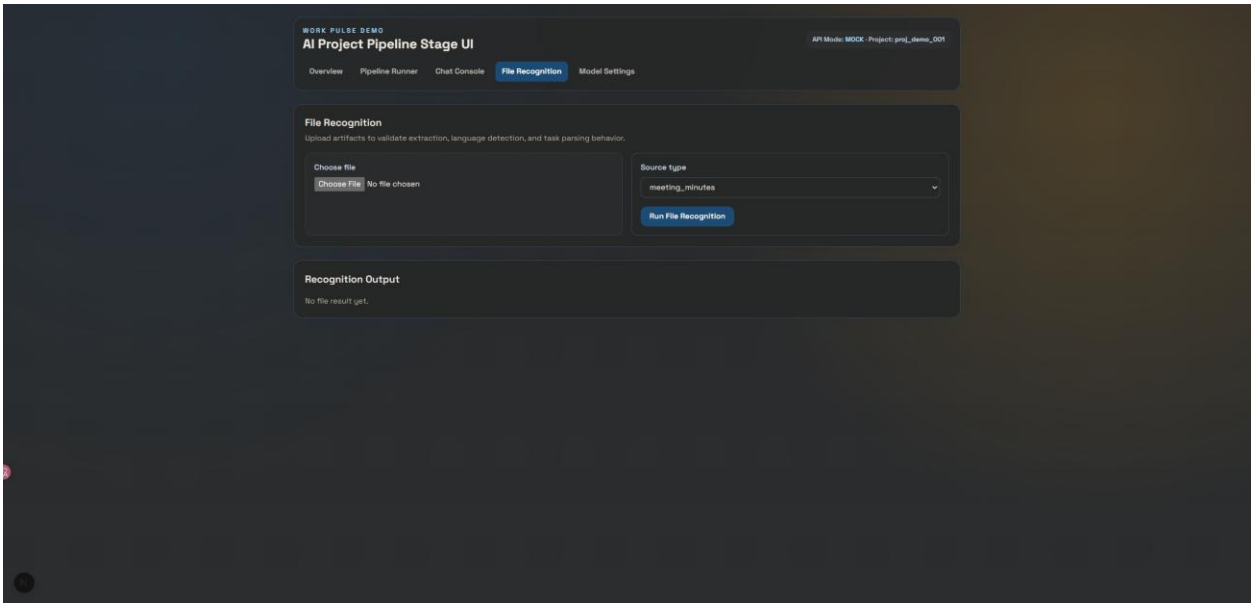


Figure 9 demo 4

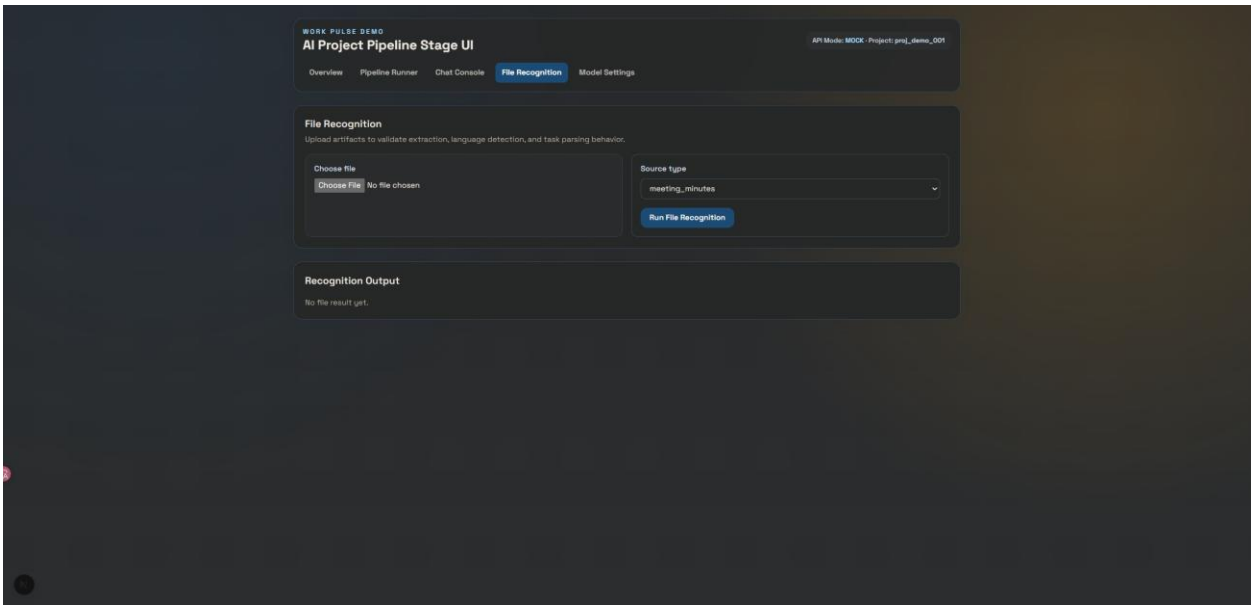


Figure 10 demo 5

3 Prototype and Test

3.1 Test data and set up

In this deliverable stage, we are still using the ai generated mock data files, but we received some anonymized data files from SSC officers. Those files includes Business Case, Project Charter, Benefits Plan, SOW, Executive Brief and meeting transcripts. These are useful as realistic, messy input sources. We made up some test data again based on these reference files to test our modules.

3.1.1 Testdata

These test files are in the folder of /testdata/. It includes four email samples in emails/, two meeting-minute samples in meeting_minutes/, one SOW baseline document in sow/, and three project status reports in status_reports/. It also includes thirteen generated artifacts in generated/, such as ground-truth JSON files (ingestion_ground_truth.json, risk_ground_truth.json, drift_ground_truth.json, priority_test_tasks.json) and edge-case inputs like cross_check_pair_a.txt and cross_check_pair_b.txt, plus several drift scenarios (within scope, scope expansion, contradiction, ambiguous, and French scope creep). Overall, this dataset design covers multi-document inputs, multilingual behavior, explicit and implicit risks, scope alignment vs scope drift, and ranking behavior under rollover conditions.

3.1.2 Hardware and Software Environment

All tests were run on a Windows machine with the following hardware:

Table 2 Test Hardware Specifications

Component	Specification
CPU	Intel Core i7-13700T
RAM	64 GB
GPU VRAM	30 GB
Operating System	Windows

We tested three tiers of language models in this deliverable, corresponding to increasing model size and capability:

Table 3 Model Tiers Used in Testing

Tier	Label	Model ID	Inference Backend	Cost
Tier 1 (Small)	**local**	**qwen/Qwen3.5-35B-A3B**	**Ollama (local, on-device)**	**Free**
Tier 2 (Medium)	**medium**	**google/gemini-3-flash**	**OpenRouter API**	**Pay-per-token**
Tier 3 (Large)	**large**	**anthropic/claude-sonnet-4.6**	**OpenRouter API**	**Pay-per-token**

We set up an OpenRouter API key for cloud-based inference (Tier 2 and Tier 3), and deployed an Ollama background service for local inference of the Tier 1 model using the machine's GPU.

We also defined two pipeline modes that apply to LLM-based methods across all modules:

- Hybrid mode: Traditional ML or regex-based preprocessing is used first to extract structure and metadata, and the LLM is then called only for higher-level reasoning tasks (e.g., task extraction, risk classification). This splits the work across two calls: one shorter, one more focused.

- Full LLM mode: A single, larger LLM call handles the entire extraction and reasoning pipeline, artifact type detection, language detection, metadata extraction, and task extraction, in one prompt. This reduces the number of API calls but increases prompt size per call.

The Traditional baseline uses no LLM at all, it relies entirely on regular expressions and rule-based parsing.

3.2 M1 Ingestion Pipeline Prototype

3.2.1 Purpose

The Ingestion Pipeline is the first module in our architecture and handles the entire input processing layer: receiving documents in various formats (email, PDF, DOCX, plain text, meeting notes), detecting their artifact type and language, extracting metadata, and producing structured task and action-item lists.

The key assumptions this prototype is designed to validate are:

1. An LLM-based approach can outperform a regex-only baseline in extracting action items from unstructured text, particularly when the ground truth requires precise recall and precision (F1 $\geqslant$ 0.80).
2. A single "full" LLM call (handling type detection, language detection, metadata, and task extraction in one prompt) can perform comparably to or better than a "hybrid" approach that splits these into two calls.
3. Local inference (Qwen3.5-35B running on Ollama) provides acceptable performance compared to cloud-based models at zero API cost.
4. The ingestion pipeline handles multilingual inputs (English and French) without degradation.

This prototype proved our assumption from Deliverable B.

3.2.2 Design & Implementation

We implemented seven configurations of the ingestion pipeline to systematically compare performance across model tiers and pipeline modes.

Table 4 Ingestion Prototype Configurations

Configuration Label	Model Tier	Pipeline Mode	LLM Backend
Traditional	None (regex only)	N/A	None
Local hybrid	Qwen3.5-35B-A3B	Hybrid	Ollama (local)
Local full	Qwen3.5-35B-A3B	Full LLM	Ollama (local)
Medium hybrid	google/gemini-3-flash	Hybrid	OpenRouter
Medium full	google/gemini-3-flash	Full LLM	OpenRouter
Large hybrid	anthropic/claude-sonnet-4.6	Hybrid	OpenRouter
Large full	anthropic/claude-sonnet-4.6	Full LLM	OpenRouter

The scoring system for the Ingestion module evaluates 15 test cases for a maximum total of 121 points. Each test case targets a specific requirement (e.g., ING-01 through ING-15). The most critical test case in this round of testing was ING-04, which evaluates action item extraction quality using an F1 score threshold of 0.80. The ground truth for ING-04 specifies exactly 5 action items; any model that extracts more than 5 will suffer a precision penalty that pushes F1 below the threshold.

3.2.3 Test Results

Table 5 IWngestion Module: Full Results by Configuration

Configuration	Score /121	Tests Passed	Tests Failed	ING-04 Score	Avg. LLM Latency	Notes
Traditional	93	12	3	0.0 FAIL	—	Regex only; fails ING-04, ING-05, ING-10
Local hybrid	119	13	2	7.7 FAIL	12,080 ms	Extracts 8 items vs. GT 5; precision too low
Local full	120	13	2	9.1 FAIL	9,644 ms	Extracts 6 items vs. GT 5

Configuration	Score /121	Tests Passed	Tests Failed	ING-04 Score	Avg. LLM Latency	Notes
Medium hybrid	118	14	1	7.1 FAIL	3,644 ms	Extracts 9 items vs. GT 5
Medium full	121	15	0	10.0 SUC	3,680 ms	Only config to extract exactly 5 items
Large hybrid	118	14	1	7.1 FAIL	6,397 ms	Cascade failure risk (see Note 1)
Large full	119	13	2	8.3 FAIL	7,629 ms	Extracts 7 items vs. GT 5

During one of our initial runs, the Large hybrid configuration returned None from the LLM call on ING-03, which caused an empty task artifact to be cached. This then caused ING-04, ING-05, and ING-06 to all receive a score of 0.0 due to reading from the corrupted cache, resulting in a misleadingly low total score of 94/121 for that run. We re-ran the Large hybrid configuration confirmed the cascade failure was intermittent: the re-run returned 118/121, with only ING-04 failing due to the standard precision issue. We treat the re-run result as the reliable figure. A fallback mechanism to prevent empty artifact caching is recommended as a future fix.

Table 6 Ingestion Module: Timing Comparison

Configuration	Avg. Single LLM Call	Full Batch (15 tests × hybrid + full)
Traditional	N/A	N/A
Local hybrid	12,080 ms	162.5 s (combined with Local full)
Local full	9,644 ms	↑ (same batch)
Medium hybrid	3,644 ms	76.0 s (combined with Medium full)
Medium full	3,680 ms	↑ (same batch)
Large hybrid	6,397 ms	116.2 s (combined with Large full)
Large full	7,629 ms	↑ (same batch)

3.2.4 Analysis

The most important finding from the Ingestion prototype is that ING-04 is the universal bottleneck: every configuration except Medium full fails it, and the failure mode is consistent, models tend to over-extract action items relative to the ground truth. This suggests our prompt for action item extraction does not adequately constrain the output quantity, and we will need to add more explicit instructions in future iterations.

The Full LLM mode consistently outperforms Hybrid mode at the same model tier. This is somewhat counterintuitive; we expected the more targeted hybrid approach (where the LLM only handles extraction) to be better. Instead, Full mode appears to benefit from having the full document context available when deciding both what to extract and how to label it. The coherence between artifact type detection and task extraction in a single call seems to improve precision.

Medium is the sweet spot for production use: it achieves the best score (121/121), the lowest cloud API latency (~3.6 s/call), and the lowest total batch runtime (76 s). Local is the best free option (120/121 at no API cost, albeit ~2× slower). Large is not justified by its cost or latency for this module.

The Large hybrid cascade failure is a reliability concern even though it appears intermittent. We recommend adding an explicit fallback in the caching layer to prevent a None LLM response from being written to the artifact cache as an empty list.

3.3 Risk Identifier Prototype

3.3.1 Purpose

The Risk Identifier module (M3) is responsible for reading project documents and communications and surfacing both explicit and implicit risks. Explicit risks are those clearly stated in the document and implicit risk require inference from context.

The key assumptions this prototype validates are:

- An LLM can surface implicit risks that regex or keyword-matching approaches would miss entirely.
- The quality of LLM-generated risk descriptions (measured by textual similarity to ground-truth descriptions using SequenceMatcher ratio $\geqslant$ 0.80) is substantially higher than template-based traditional outputs.
- Local and cloud models both achieve acceptable risk identification quality, validating the feasibility of a cost-free deployment option.

3.3.2 Design and Implementation

The Risk module was tested in four configurations. Unlike Ingestion, we did not implement a "full" LLM mode variant for Risk in this iteration, only the hybrid pipeline (traditional preprocessing + LLM reasoning) and the traditional baseline were tested. Full mode for Risk is planned for a future iteration.

The scoring system evaluates 13 test cases for a maximum of 104 points. The key evaluation metric for risk description quality is a Sequence Matcher similarity ratio, with a threshold of 0.80, meaning the LLM-generated risk description must be at least 80% similar to the reference description in the ground truth.

3.3.3 Test Results

Table 7 Risk Identifier Module: Results by Configuration

Configuration	Score /104	Tests Passed	Tests Failed	Key Failures	Notes
Traditional	76	10	3	RSK-01, RSK-02, RSK-06	Template outputs; similarity ratio ~0.34, far below 0.80
Local hybrid	104	13	0	—	Perfect score
Medium hybrid	100	12	1	RSK-05	Hallucination detected in risk description
Large hybrid	104	13	0	—	Perfect score

3.3.4 Analysis

The Risk module shows the clearest gap between traditional and LLM-based approaches. The traditional method fails RSK-01 and RSK-02 not because it identifies the wrong risks, but because its output descriptions are templated and formulaic — the SequenceMatcher ratio against

the ground truth's more natural-language descriptions sits around 0.34, which is less than half the 0.80 threshold. This finding validates our core design decision to use an LLM for risk description generation rather than a lookup table or template system.

Both Local and Large achieve perfect scores (104/104), which is encouraging: it confirms that the free local model (Qwen3.5-35B) is sufficient for risk identification quality, and we do not necessarily need to spend on API calls for this module in production.

Medium's failure on RSK-05 (hallucination check) is worth noting. The hallucination was detected by our automated evaluator, which compares the model's cited source context against the actual document. This suggests gemini-3-flash may be more prone to fabricating risk context under certain prompt configurations, and we should add an explicit grounding instruction to the Risk module prompt for Medium.

3.4 Priority Ranker Prototype

3.4.1 Purpose

The Priority Ranker module receives a list of tasks, potentially including rolled-over tasks from previous days, and assigns priority scores or rankings based on contextual signals: deadlines, associated risk factors, project criticality, and task interdependencies.

The key assumptions this prototype validates are:

- An LLM can rank tasks more meaningfully than a simple rule-based system by incorporating contextual signals (e.g., inferring urgency from meeting tone, not just deadlines).
- The LLM-based ranker produces consistent rankings for the same input across repeated calls (testing for output stability).
- Hallucination in priority justifications can be detected programmatically, and the rate of hallucination is low enough to be acceptable in production.

3.4.2 Design and Implementation

The Priority module was tested across four configurations. The Traditional baseline was only evaluated on 9 of the 12 test cases, specifically those for which we had pre-defined rule-based

priority baselines. The remaining 3 test cases require contextual inference that the traditional system cannot perform, so they were only evaluated for LLM methods.

The scoring system evaluates up to 12 test cases for a maximum of 97 points. The critical test case in this module was PRI-05, which checks for hallucinated justifications: if the model provides a priority explanation that cites a deadline, risk, or task relationship that does not exist in the input, the test fails.

3.4.3 Results

Table 8 Priority Ranker Module: Results by Configuration

Configuration	Score /97	Tests Passed	Tests Failed	Key Failures	Notes
Traditional	72	9	0	—	Only 9 cases run; 0 failures but limited coverage
Local hybrid	91	11	1	PRI-05	hallucinations = 3 detected
Medium hybrid	94	11	1	PRI-05	Best LLM result; unexpectedly better than Large
Large hybrid	82	10	2	PRI-05, PRI-10	Large fails PRI-10 (consistency), Medium does not

3.4.4 Analysis

The Priority Ranker results contain two particularly interesting findings.

First, PRI-05 (hallucination detection) fails across all LLM configurations. This is our most significant unresolved quality issue in this module. The current prompt does not include explicit grounding instructions (e.g., "only cite deadline or risk information that is explicitly present in the task list, do not infer or fabricate values"), and all three model tiers produce at least some

hallucinated justifications. This is a known issue and is flagged for prompt engineering work in the next iteration.

Second, and somewhat surprisingly, Medium hybrid (gemini-3-flash) outperforms Large hybrid (claude-sonnet-4.6) in this module. The Large model scores 82/97, a full 12 points below Medium's 94/97, because it additionally fails PRI-10, the consistency test. This means that for the same input, the Large model produces different top-3 priority rankings across repeated calls, while Medium does not. We believe this is related to higher output temperature or sampling variance in the Claude model configuration we used, and it could be tuned. However, it also suggests that model size alone does not guarantee better results for structured output tasks that require consistency.

However, we are not sure if these kind of ranking aligns clients' requirements. We will discuss and make a future adjustment.

3.5 Drift Detection Prototypes

3.5.1 Purpose

The Drift Detection module (M5) continuously monitors project communications and compares them against the original Statement of Work (SOW) baseline. When new requests or updates arrive, the module classifies them into one of several scope categories: within scope, scope expansion (out-of-scope request), contradiction (conflicting with SOW), ambiguous (borderline case), or French scope creep (French-language scope deviation).

The key assumptions this prototype validates are:

- An LLM can correctly classify scope alignment across all five scenario types, including the edge cases (ambiguous and contradiction).
- The system correctly handles French-language scope creep scenarios without degradation.
- Traditional rule-based approaches are fundamentally insufficient for Drift Detection given the semantic complexity of scope classification.

3.5.2 Design and Implementation

Drift Detection presents a unique testing challenge: most of the test scenarios we defined are inherently LLM-only, they require semantic understanding that traditional methods simply cannot provide. As a result, the Traditional baseline was only evaluated on 2 of the 13 test cases (those for which a simple keyword-based rule could be defined), giving Traditional a maximum possible score of approximately 16/104 under fair comparison conditions.

Table 9 Drift Detection Prototype Configurations

Configuration	Model	Pipeline Mode	Test Cases Run	Notes
Traditional	None	Keyword-based	2 of 13	Only test cases with keyword-matchable ground truth
Local hybrid	Qwen3.5-35B-A3B	Hybrid	13 of 13	—
Medium hybrid	google/gemini-3-flash	Hybrid	13 of 13	—
Large hybrid	anthropic/claude-sonnet-4.6	Hybrid	13 of 13	—

The five scenario types in the test dataset, and their design intent, are summarized below.

Table 10 Drift Detection Scenario Types

Scenario File	Type	Design Intent
drift_within_scope.txt	Within scope	Negative case: should NOT trigger alert
drift_scope_expansion.txt	Scope expansion	Clear out-of-scope request; should trigger alert
drift_contradiction.txt	Contradiction	Conflicting instructions; should flag as contradiction
drift_ambiguous.txt	Ambiguous	Borderline; model must handle uncertainty gracefully

Scenario File	Type	Design Intent
drift_french_scope_creep.txt	French scope creep	French-language SOW deviation; bilingual robustness test

3.5.3 Results

Table 11 Drift Detection Module: Results by Configuration

Configuration	Score /104	Tests Passed	Tests Failed	Key Failures	Notes
Traditional	12	2	0	—	Only 2 tests with keyword-based baselines; score is not comparable
Local hybrid	82	10	3	DFT-03, DFT-05, DFT-06	Scope boundary edge cases
Medium hybrid	78	9	4	DFT-03, DFT-05, DFT-06, DFT-07	One additional failure vs Local/Large
Large hybrid	82	10	3	DFT-03, DFT-05, DFT-06	Same failures as Local

3.5.4 Analysis

Drift Detection is our most difficult module to get right which is exactly the same with what we thought, and this prototype confirms that. The common failure cluster, DFT-03, DFT-05, and DFT-06, represents exactly the types of cases that make Drift Detection interesting: ambiguous boundary decisions and contradiction detection require nuanced contextual reasoning that even large LLMs struggle with in their current prompt configuration.

Notably, all three LLM tiers fail the same three test cases, which tells us this is not a model capacity problem but a prompt design problem. Our current prompt asks the model to classify scope alignment but does not give it explicit examples of what constitutes an ambiguous boundary versus a clear expansion. We plan to add few-shot examples for these edge case categories in the next iteration.

The French scope creep test (DFT-06) failing across all methods is a particular concern, given that bilingual support is one of our core value propositions for the SSC context. We believe this may be due to our SOW baseline document being in English only, making it harder for the model to reason about French-language deviations. A bilingual SOW baseline would likely improve this.

3.6 Cross-Module Summary and Integration Testing

3.6.1 Overall Score Summary

The table below summarizes performance across all four modules and all tested configurations.

The local small-scale model works surprisingly good with hybrid implementation, and the traditional way shows the lack of the detecting methods of some of the tasks.

Table 12 Overall Cross-Module Test Results

Configuration	Ingestion /121	Risk /104	Priority /97	Drift /104	Total /426
Traditional	93 (12P / 3F)	76 (10P / 3F)	72 (9P / 0F)[1]	12 (2P / 0F)[2]	253
Local hybrid	121 (15P / 0F)	104 (13P / 0F)	91 (11P / 1F)	82 (10P / 3F)	398
Medium hybrid	118 (14P / 1F)	100 (12P / 1F)	94 (11P / 1F)	78 (9P / 4F)	390
Large hybrid	118 (14P / 1F)	104 (13P / 0F)	82 (10P / 2F)	82 (10P / 3F)	386

3.6.2 Known Issues Summary

The following table consolidates all known issues identified during this round of testing:

Issue	Affected Module	Affected Tests	Root Cause	Current Status	Proposed Fix
Large hybrid LLM returns None, corrupts dedup cache	Ingestion	ING-04, ING-05, ING-06 (cascade)	LLM call failure not caught before cache write	Intermittent; re-run confirmed	Add None / empty-response fallback before caching
ING-04 action item over-extraction	Ingestion	ING-04 (all configs)	Prompt does not constrain output count	Only Medium full passes	Add explicit count constraint to extraction prompt
PRI-05 hallucination in priority justifications	Priority	PRI-05 (all LLM)	No grounding instruction in prompt	Unresolved	Add "cite only facts present in input" instruction
DFT-03/05/06 scope boundary misclassification	Drift	DFT-03, 05, 06 (all configs)	No few-shot examples for edge cases in prompt	Unresolved	Add few-shot examples for ambiguous / contradiction classes
DFT-06 French scope creep failure	Drift	DFT-06 (all configs)	English-only SOW baseline; bilingual reasoning degraded	Unresolved	Provide bilingual (EN/FR) SOW baseline for comparison
PRI-10 Large hybrid ranking inconsistency	Priority	PRI-10 (Large only)	High output variance in claude-sonnet-4.6	Unresolved	Reduce temperature / add deterministic output formatting

4 Updated Prototype Test Plan

4.1 Current Status

At this stage of the project, we have moved from architecture planning into a working prototype phase with measurable test evidence. We completed the first full implementation of the core backend services. We also completed the unified test harness and generated benchmark outputs across Traditional and LLM-based methods (local, medium, and large tiers).

The main unresolved problems are also consistent across the evidence:

- Drift boundary errors (especially contradiction vs expansion, plus ambiguous cases).
- Priority reasoning hallucinations in some LLM runs.
- Ingestion precision instability in specific extraction cases (for example, over-extraction behavior in stricter F1 tests).
- Reliability hardening needs, such as safer fallback behavior after failed LLM calls.

At the same time, we identified two strategic additions for the next cycle. First, we need an email bot integration path so project teams can forward real workflow emails directly into the ingestion pipeline without manual copy-paste. Second, we want to test a new open-source local fine-tuned file-processing model that supports anonymized data workflows. This is highly relevant because our target clients have strict and complex security requirements, and local anonymized processing may offer a better governance fit than cloud-only inference.

It is important to clarify that our current web UI is a test UI, not the final client-facing product. We are using it as an internal validation tool to exercise module behavior and expose defects quickly. Before building the next UI version, we need a structured client review cycle to confirm whether module outputs (risk evidence quality, drift classification quality, and ranking usefulness) actually match stakeholder expectations.

In short, our prototype backend is functional, but the product workflow is still in a validation loop: test UI -> client feedback -> module revision and re-test -> new UI -> full pipeline acceptance test.

4.2 Project Plan Updates

Compared with our earlier plan, we adjusted priorities to protect delivery quality under time and resource constraints. We are keeping the module scope (M1-M5), but sequencing the work more strictly and in a more client-centered order:

First, we use the current test UI to run guided module demonstrations with clients and collect expectation-level feedback.

Second, we revise module logic based on that feedback and re-run targeted tests on known weak cases.

Third, we add an email bot integration workstream and run a controlled trial of a local anonymization-friendly fine-tuned file model for secure document handling.

Fourth, we build a new UI for delivery readiness (separate from the current test UI).

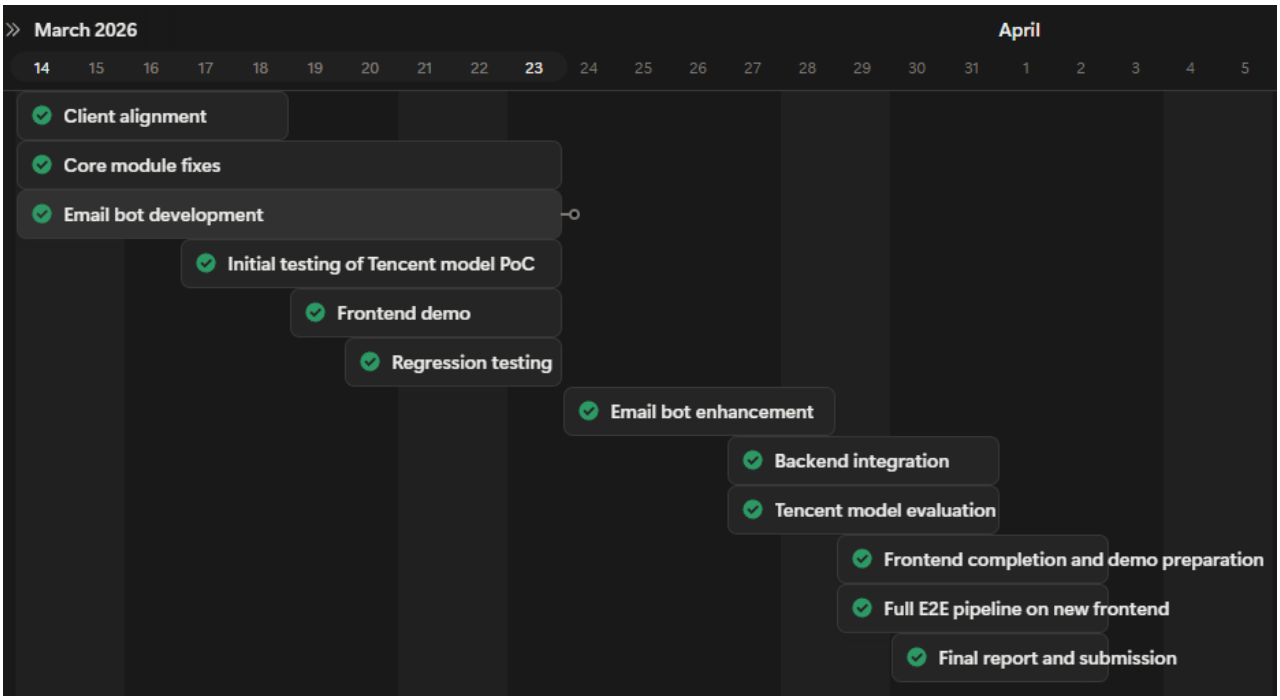
Finally, we run a full pipeline test in the new UI to confirm end-to-end quality.

This update also reflects a practical student-team decision: we prioritize quality of evidence over broad feature inflation. Features like full auth/RBAC, deeper deployment automation, and non-critical UX extras remain deferred unless core test quality targets are met first.

4.3 Task Assignments

Member	Primary Focus (Next 4 Weeks)
Zhenyan Xing (Backend Integration Lead)	Stabilize M1-M4 integration flow, implement email bot integration endpoint path, fix fallback and error propagation
Zhe Wang (Risk + Drift QA Lead)	Improve M3/M5 prompt behavior, redesign edge-case test sets, and evaluate secure anonymized local model behavior on sensitive-like inputs
Aoqi Liu (Frontend + Demo Lead)	Maintain current test UI for client walkthroughs, then build a new production-style UI and run full pipeline verification
Lizzy-Preiz (Test Harness + Reporting Lead)	Maintain test.py workflow, consolidate score summaries, write report updates and evidence tables

4.4 Time Schedule



4.5 Contingency Plan Updates

4.5.1 API or model instability

If external model calls fail or become unstable(such as latency becomes too high), we immediately switch affected demonstrations to local-tier or mock-assisted flow while preserving the same pipeline structure. We keep Traditional mode as a fallback baseline for comparison, not as the main demo path.

4.5.2 Transition risk from test UI to new UI

Because the current interface is only a test UI, we will not treat it as a final deliverable. If new UI development slips, we will still complete module-level validation first, then lock a minimal release candidate UI that supports full pipeline testing before adding visual refinements.

4.5.3 Security requirement escalation from clients

If client security requirements become stricter during review, we will prioritize the local anonymization-capable model trial and only allow cloud-tier calls for non-sensitive test datasets. This gives us a practical migration path toward higher compliance without blocking ongoing module validation.

4.5.4 Drift module confidence ceiling risk

If the Drift module cannot be revised to a sufficiently high confidence level after targeted prompt and test iterations, we will formally re-evaluate its role in the product scope. Our fallback options are: (a) downgrade Drift from an auto-decision module to a decision-support signal with mandatory human review, or (b) narrow Drift to high-confidence scenario types only (for example, obvious scope expansion) and remove low-confidence categories from release claims. This prevents us from overpromising governance reliability.

4.5.5 Tencent local anonymization model substitution risk/opportunity

Tencent has released a new local-processing LLM that claims anonymization capability. Because our clients have strict security requirements, we will run a structured feasibility study to determine whether this model can partially or fully replace the current Ingestion module path.

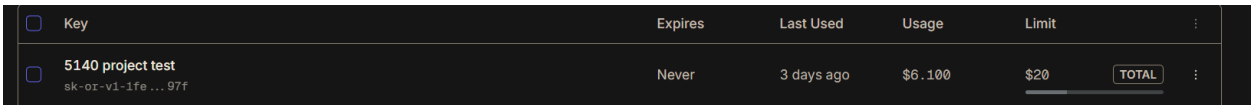
The study will compare extraction quality, anonymization reliability, latency, integration effort, and operational risk. If results are strong, we will design a migration option where sensitive document ingestion is handled by the local anonymization model, while other modules continue in the existing pipeline.

5 Bill of Materials

During the current development and testing phase, our primary expenses come from using APIs to test our models. However, moving forward, we will definitely need to rent a server to provide a shared testing platform for all team members, as well as to demonstrate our work to clients and classmates. Another ongoing cost is the electricity consumption from running and testing models locally, though this is quite difficult to quantify.

5.1.1 API

So far, our testing has cost around $6. We are using OpenRouter APIs to evaluate various models. However, since the documents we are currently testing have very limited word counts, we anticipate our future costs to increase by at least tenfold. Therefore, we are allocating $60 to this portion of the budget.



Key	Expires	Last Used	Usage	Limit
5140 project test sk-or-v1-1fe...97f	Never	3 days ago	$6.100	$20 TOTAL

Figure 11 Current api cost

5.1.2 Server renting

So far, our testing has cost around $6. We are using OpenRouter APIs to evaluate various models.

Cloud Provider	Setup Tier	Instance Name	Compute Specs	Storage (Boot Disk)	Total Est. Monthly Cost (USD)
GCP	**Cost-Effective**	e2-medium	2 vCPU, 4GB RAM (Shared-core)	30GB Balanced SSD	**~$27.46**
GCP	**Performance**	e2-standard-2	2 vCPU, 8GB RAM (Dedicated)	30GB Performance SSD	**~$54.01**
Azure	**Cost-Effective**	Standard_B2s	2 vCPU, 4GB RAM (Burstable)	32GB Standard SSD	**~$32.77**
Azure	**Performance**	Standard_D2s_v5	2 vCPU, 8GB RAM (General Purpose)	32GB Premium SSD	**~$77.28**

5.1.3 Electricity

Because we have been running models locally, the prolonged operation consumes a significant amount of electricity. However, since this is somewhat difficult to track and calculate accurately, we are temporarily excluding it from our total budget.

6 Conclusions and Recommendations for Future Work

During the current development and testing phase, our primary expenses come from using APIs to test our models. However, moving forward, we will definitely need to rent a server to provide a shared testing platform for all team members, as well as to demonstrate our work to clients and classmates. Another ongoing cost is the electricity consumption from running and testing models locally, though this is quite difficult to quantify.

This deliverable demonstrates that our team has successfully moved from problem analysis and system design into a functional prototype stage for an AI-powered project management assistant tailored to SSC-like environments. The proposed platform addresses the core challenge of fragmented project information spread across emails, meeting notes, and local files by integrating five main modules: Ingestion Pipeline, Priority Ranker, Risk Engine, Daily Planner, and Drift Detector. Together, these modules aim to support daily work orchestration, risk identification, scope control, and legacy information management in a more structured and evidence-driven workflow.

The testing results show that LLM-based approaches clearly outperform traditional rule-based methods for higher-level reasoning tasks, especially in risk identification and semantic interpretation. In the detailed module results, the Medium full configuration achieved the strongest ingestion performance, while the local and large hybrid configurations achieved perfect scores in the Risk module. At the same time, the evaluation also revealed several important weaknesses: action-item over-extraction in ingestion, hallucinated justifications in priority ranking, and persistent errors in drift classification, especially for ambiguous scope boundaries and French scope-creep scenarios. These findings confirm that the prototype backend is already functional, but its reliability still depends heavily on prompt design, grounding quality, and edge-case handling rather than model size alone.

A key lesson learned from this deliverable is that better results do not always come from using the largest model. Instead, performance depends on choosing the right model for the task, designing prompts carefully, and validating outputs with structured test cases. The project also showed that local models are promising for secure and cost-effective deployment, which is particularly important in government-style settings with strict data governance requirements. However, the current system is still in a validation loop, and the present interface should be understood as a test UI rather than a final client-facing product.

For future work, the team should focus on improving prompt grounding and few-shot guidance, especially for priority justification and drift detection. A bilingual SOW baseline should be introduced to improve French scope-creep detection, and stronger fallback logic should be added to prevent cache corruption or unstable model responses. In parallel, the next development cycle should include structured client review sessions, email bot integration for real workflow ingestion, evaluation of a local anonymization-capable fine-tuned model, and the development of a new production-style UI followed by full end-to-end acceptance testing. By prioritizing evidence quality, security alignment, and client feedback over feature expansion, the project can progress toward a more robust, trustworthy, and deployment-ready intelligent assistant.

7 Bibliography

[1] Government of Canada, "2024-25 Departmental Plan: Raison d'être, mandate and role," Shared Services Canada, 2024. [Online]. Available: https://www.canada.ca/en/shared-services/corporate/about-us/transparency/departmental-plan/2024-25/2024-25-departmental-plan/raison-etre-mandate-role.html

[2] Government of Canada, "Canada's Digital Ambition 2024-25," Treasury Board of Canada Secretariat, 2024. [Online]. Available: https://www.canada.ca/en/government/system/digital-government/canada-digital-ambition/canada-digital-ambition-2024-25.html

[3] Government of Canada, "Canada's Digital Ambition 2023-24," Treasury Board of Canada Secretariat, 2023. [Online]. Available: https://www.canada.ca/en/government/system/digital-government/canada-digital-ambition/canada-digital-ambition-2023-24.html

[4] Shared Services Canada, "Project Management & Delivery Branch Overview," Internal Documentation, 2026.

[5] Shared Services Canada, "Project Governance and Operating Framework (PGOF)," Version 2025-03, 2025.